



Build a SOC Lab with Wazuh



DETECTION ENGINEERING • AI-ASSISTED TRIAGE • HUMAN OVERSIGHT

From Traditional SIEM Monitoring to a Human-in-the-Loop Hybrid SOC Workflow

Ayinde Perouza

Environment	VMware Workstation / VMnet8 10.1.10.0/24
Core Platform	Wazuh - Windows/Linux telemetry, FIM, custom rules, Active Response
AI Extension	Ollama + Qwen 2.5 + Python policy/validation pipeline
Report Basis	Investigation Structure + Collected Lab Evidence

Portfolio Technical Case Study | August 2026

Table of Contents

Navigation guide to the original Wazuh SOC build, investigation, independent AI-assisted extension and final evidence package.

Section	Page	Section	Page
1. Executive Summary	3	12. Recommendations	20
2. Original Project Scope and Roadmap	4	13. Project Enhancement - AI-Assisted SOC Triage	21
3. Original SOC Lab Architecture	5	14. AI Implementation Progression	22
4. Wazuh Deployment, Agents and Telemetry	6	15. AI Issues, Troubleshooting and Design Resolutions	23
5. Dashboard Development and Detection Engineering	8	16. AI Test Cases and Human Ground Truth	25
6. File Integrity Monitoring	11	17. AI Benchmark Results	26
7. SSH Abuse Detection and Active Response	14	18. Final Operational Model	27
8. Investigation Structure + Collected Lab Evidence	16	19. Lessons Learned and Skills Demonstrated	28
9. Findings	17	20. Evidence and Artifact Index	29
10. Investigation Summary and Master Evidence Timeline	18	21. References & Credits	30
11. Who, What, When, Where, Why and How	19		

Investigation Structure + Collected Lab Evidence

Findings -> Investigation Summary -> Who / What / When / Where / Why / How -> Recommendations. Questions used to drive the investigation: What happened? When did it happen? Which hosts were involved? Which accounts were changed? Was a file deleted? Was SSH abuse observed? What should be recommended?

1. Executive Summary

The project began as a hands-on Wazuh Security Operations Center lab designed to build practical experience in security monitoring, alerting, File Integrity Monitoring (FIM), custom detection rules, Active Response and incident investigation. The environment collected telemetry from Windows and Linux endpoints and used Wazuh as the central monitoring and analysis platform.

After the original investigation workflow was completed, the project was extended with a local AI-assisted Tier 1 SOC triage capability. Testing showed that a language model should not be treated as the authority for deterministic security state. A Python policy engine, output validator and human ground-truth benchmark were therefore added to create a controlled human-in-the-loop architecture.

- Functional Wazuh server with Windows and Linux agents.
- Windows Sysmon and Linux telemetry integrated into Wazuh.
- Custom SOC dashboards for severity, account-management and SSH activity.
- Real-time FIM on multiple Windows and Linux paths.
- Custom rules for Guest account enablement, Remote Desktop Users membership changes and repeated SSH failures.
- Active Response using firewall-drop to block the source of repeated SSH authentication failures.
- Formal evidence timeline and investigation using the Investigation Structure + Collected Lab Evidence.
- Local Ollama/Qwen benchmark across four SOC test cases with deterministic policy enforcement and output validation.

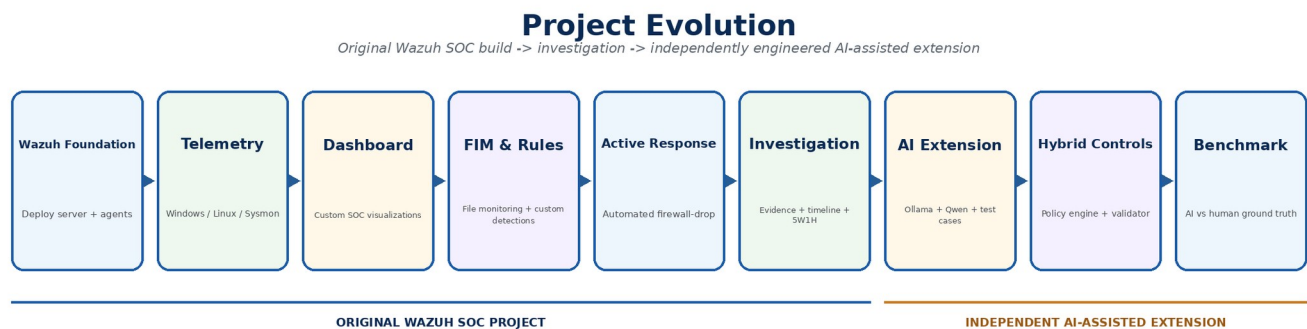


Figure 1 - Project progression from the original Wazuh SOC build through investigation and the independently engineered AI-assisted extension.

2. Original Project Scope and Roadmap

The original project objective was to build a beginner-friendly, on-premises SOC lab using virtual machines and Wazuh. The progression moved from platform fundamentals through deployment, endpoint integration, telemetry validation, dashboard development, FIM/custom rules, Active Response and incident investigation.

Phase	Original objective	Completed implementation
1 - Fundamentals	Understand Wazuh architecture and SOC role	Wazuh components, agents, indexer, dashboard and manager roles reviewed.
2 - Deployment	Deploy Wazuh and prepare telemetry pipeline	Dedicated all-in-one Wazuh server configured at 10.1.10.50.
3 - Endpoint Integration	Connect Windows and Linux endpoints	Windows agent ID 001 and Linux agent ID 002 active.
4 - Telemetry Validation	Generate and review security events	Windows Security, Sysmon, Linux journald and archive events verified.
5 - Dashboard Development	Build analyst-focused visualizations	Custom SOC Operations Dashboard built for alert severity, account events and SSH activity.
6 - Detection & FIM	Monitor files and create custom detections	Windows/Linux real-time FIM plus custom rules 100100 and 100101.
7 - Active Response	Automatically respond to suspicious activity	SSH correlation rule 100102 triggered firewall-drop against 10.1.10.51.
8 - Investigation	Investigate and document a security event	Evidence timeline, findings, 5W1H assessment and recommendations produced.

3. Original SOC Lab Architecture

The original Wazuh architecture used a dedicated Wazuh server and monitored Windows/Linux endpoints on VMware VMnet8. ADC1 supplied lab Active Directory, DNS and DHCP services, while Kali remained isolated for security testing. The auxiliary Ubuntu system at 10.1.10.52 was later reused as the AI/Ollama host.

SOC Lab with Wazuh – Network Diagram

Virtualized Lab Environment (VMware Workstation / VMnet8)

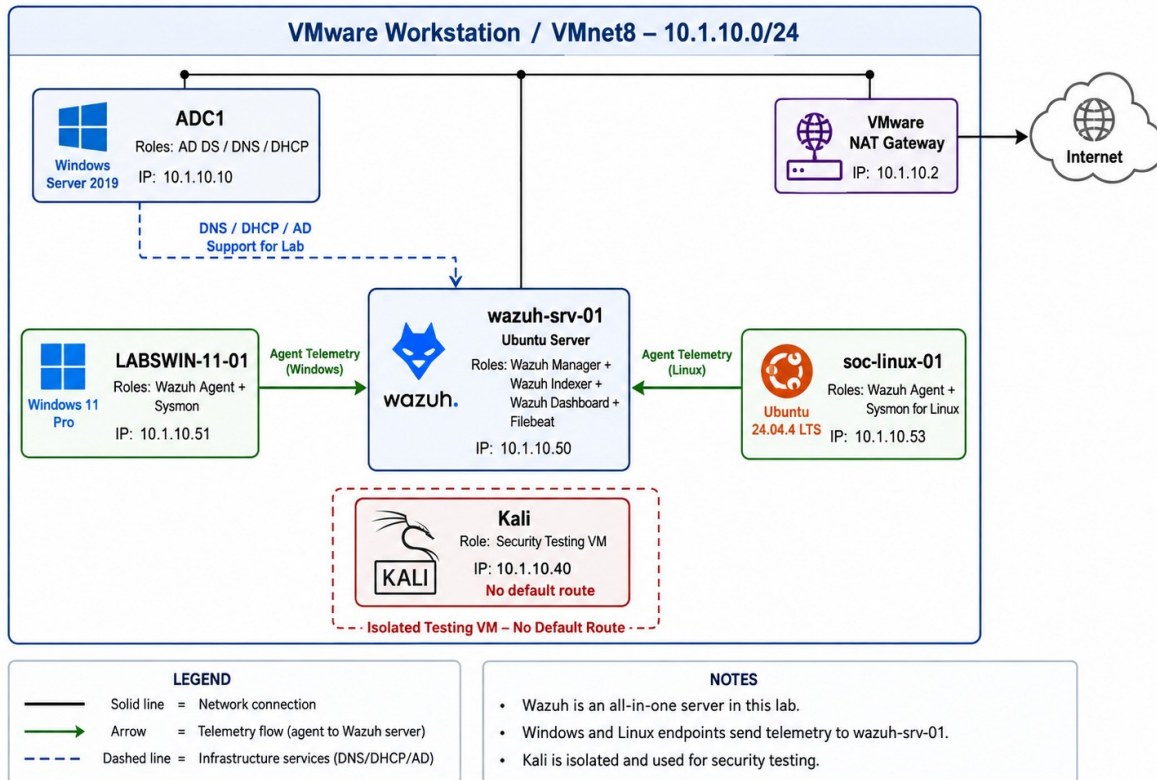


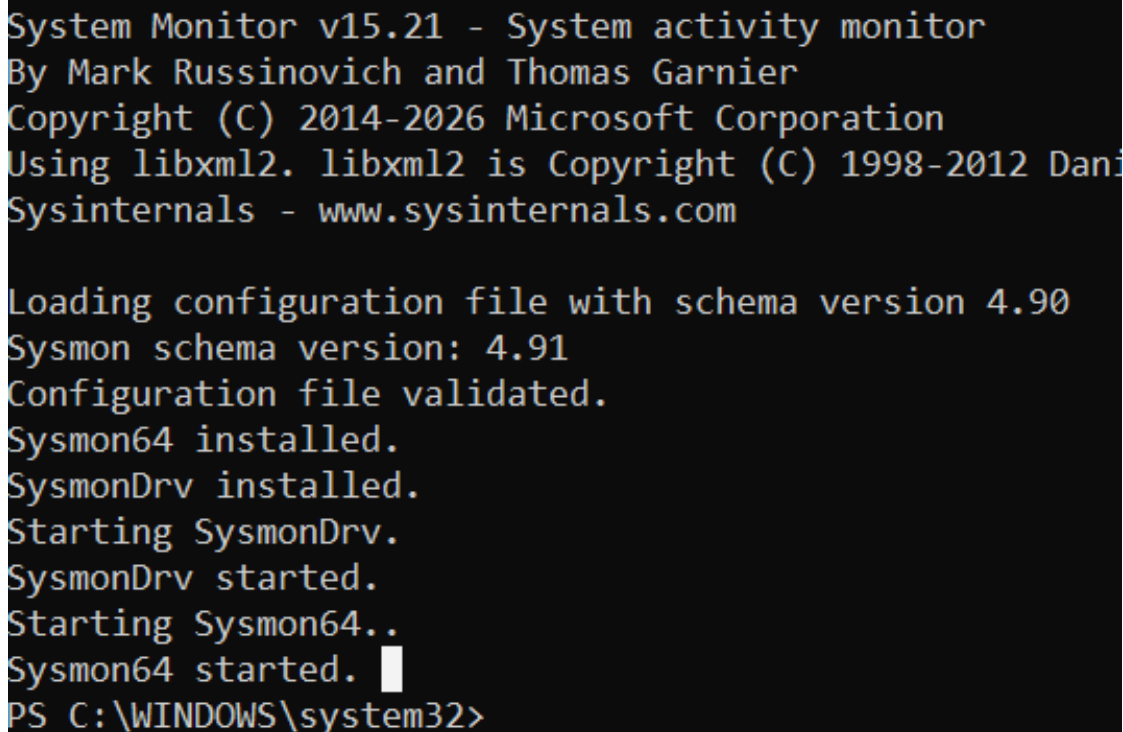
Figure 2 - Original Wazuh SOC lab network diagram. The dedicated Wazuh server is 10.1.10.50; monitored endpoints are 10.1.10.51 and 10.1.10.53.

System	IP	Role
ADC1	10.1.10.10	Windows Server 2019 - AD DS / DNS / DHCP
wazuh-srv-01	10.1.10.50	Wazuh Manager / Indexer / Dashboard / Filebeat
LABSWIN-11-01	10.1.10.51	Windows 11 endpoint - Wazuh Agent + Sysmon
wcds-app-01 / Ubuntu-Server 2	10.1.10.52	Auxiliary Ubuntu host; later local AI/Ollama platform
soc-linux-01	10.1.10.53	Ubuntu endpoint - Wazuh Agent + Linux Sysmon / SSH / FIM
Kali	10.1.10.40	Isolated security-testing VM; no default route
VMnet8 gateway	10.1.10.2	VMware NAT gateway

4. Wazuh Deployment, Agents and Telemetry

The dedicated Wazuh server was configured as the central collection and analysis platform. Windows and Linux agents were enrolled and validated. Sysmon increased endpoint visibility, and Wazuh archives were enabled to support detailed investigation and historical event hunting.

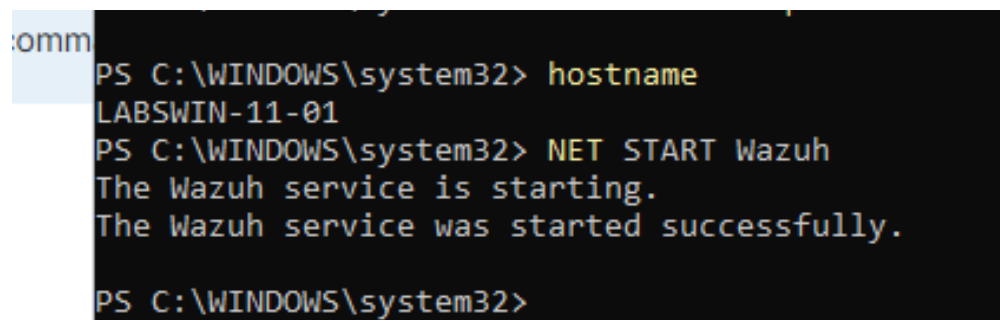
- Windows endpoint: LABSWIN-11-01, agent ID 001, 10.1.10.51.
- Linux endpoint: soc-linux-01, agent ID 002, 10.1.10.53.
- Wazuh server: wazuh-srv-01, 10.1.10.50.
- Agent event communication validated on TCP 1514; enrollment on TCP 1515.
- Dashboard accessed over HTTPS 443; indexer/OpenSearch local service on 9200.
- Sysmon 15.21 installed on Windows; Linux Sysmon and journald telemetry also observed.



```
System Monitor v15.21 - System activity monitor
By Mark Russinovich and Thomas Garnier
Copyright (C) 2014-2026 Microsoft Corporation
Using libxml2. libxml2 is Copyright (C) 1998-2012 Daniel Veith
Sysinternals - www.sysinternals.com

Loading configuration file with schema version 4.90
Sysmon schema version: 4.91
Configuration file validated.
Sysmon64 installed.
SysmonDrv installed.
Starting SysmonDrv.
SysmonDrv started.
Starting Sysmon64..
Sysmon64 started.
PS C:\WINDOWS\system32>
```

Figure 3 - Windows Sysmon installation and service startup validation on LABSWIN-11-01.



```
PS C:\WINDOWS\system32> hostname
LABSWIN-11-01
PS C:\WINDOWS\system32> NET START Wazuh
The Wazuh service is starting.
The Wazuh service was started successfully.
PS C:\WINDOWS\system32>
```

Figure 3A - Wazuh Windows agent service startup validation on LABSWIN-11-01.

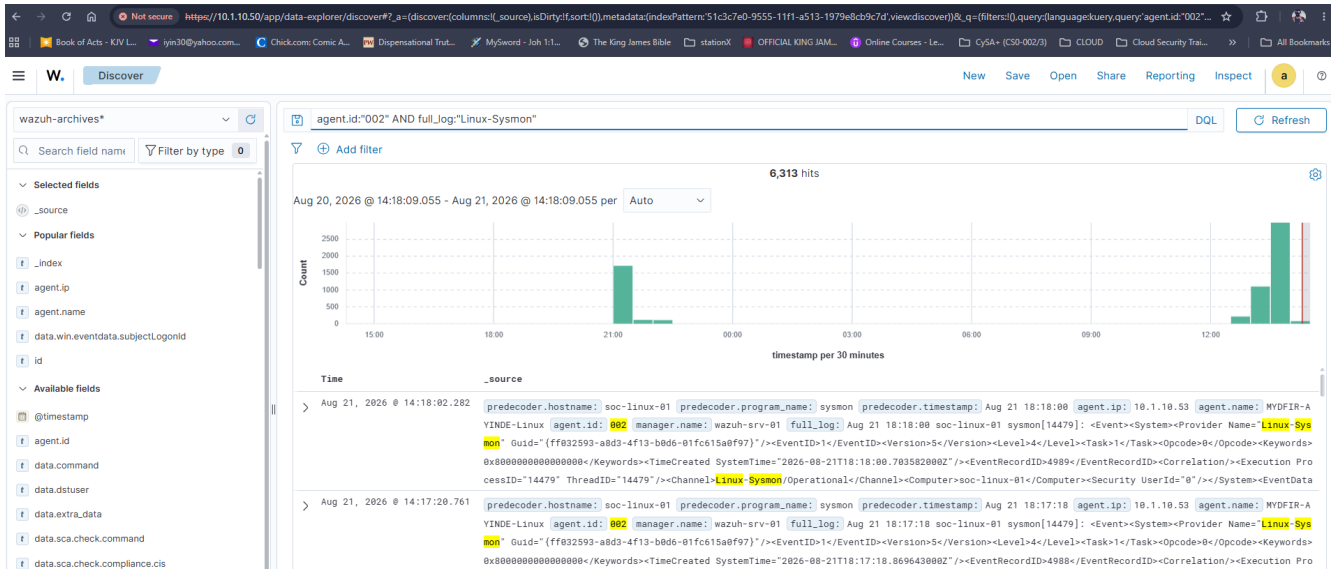


Figure 3B - Linux Sysmon telemetry from soc-linux-01 visible in Wazuh archives, confirming Linux endpoint collection.

5. Dashboard Development and Detection Engineering

The SOC Operations Dashboard was developed to make triage faster by surfacing alert volume, severity, account-management activity and failed SSH authentication patterns. Detection engineering then moved beyond built-in rules to custom rules tied to specific behaviors observed in the lab.

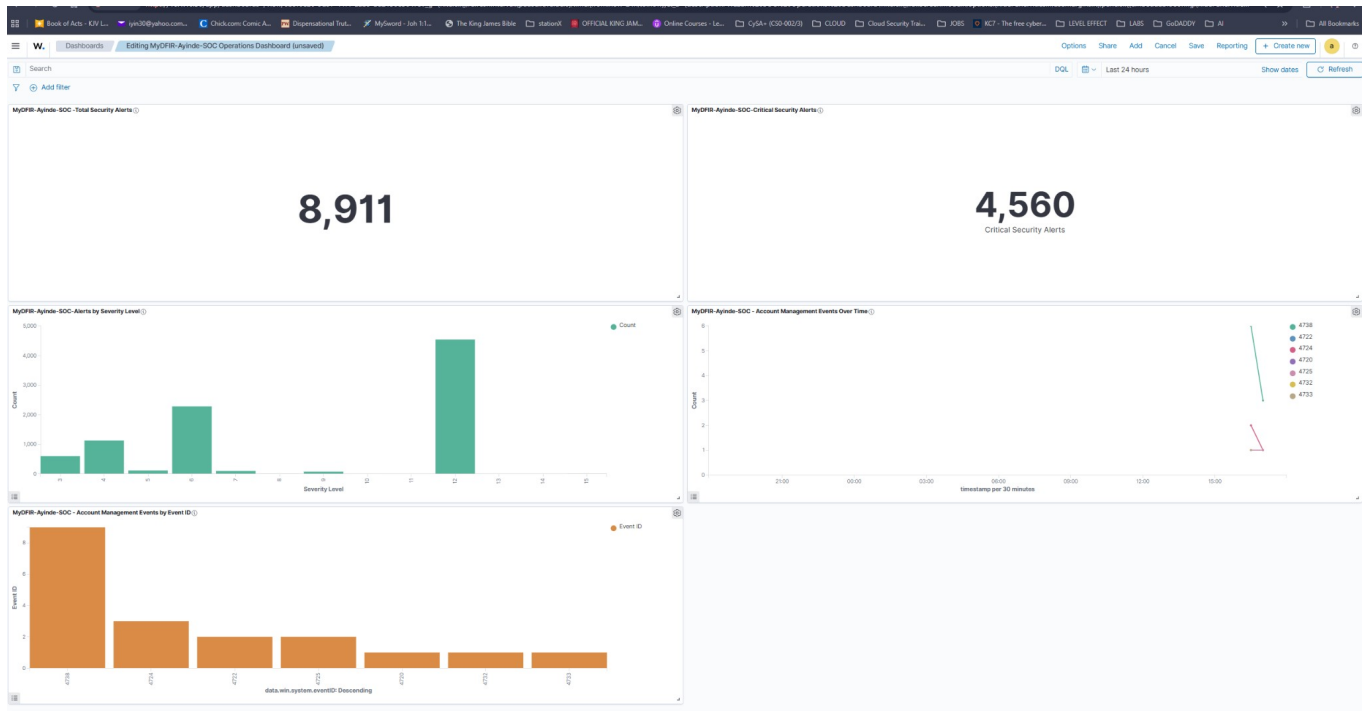


Figure 4 - Custom MyDFIR-Ayinde SOC Operations Dashboard showing total/critical alerts, severity distribution and account-management activity.

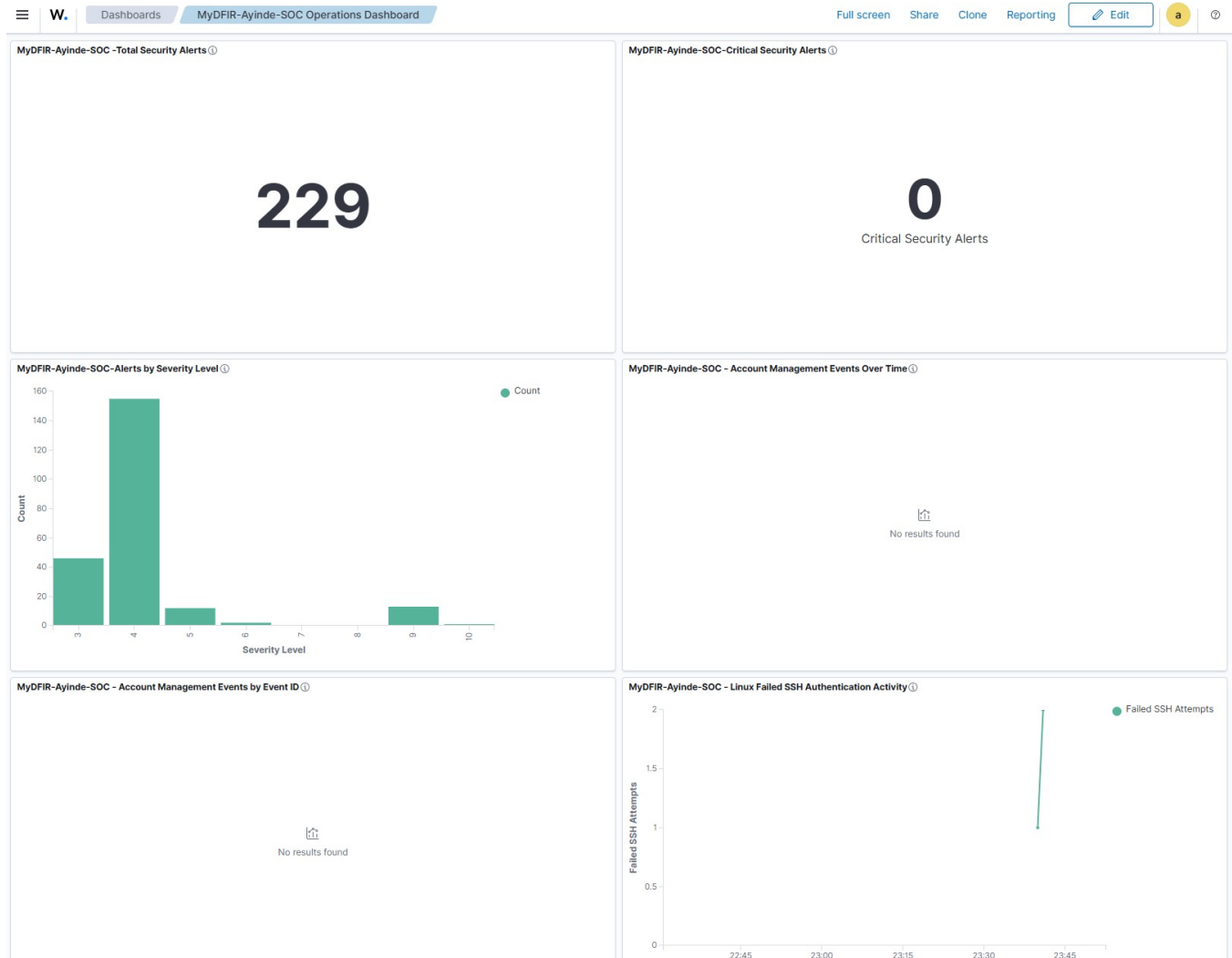


Figure 4A - Additional custom dashboard view including Linux Failed SSH Authentication Activity alongside alert and account-management panels.

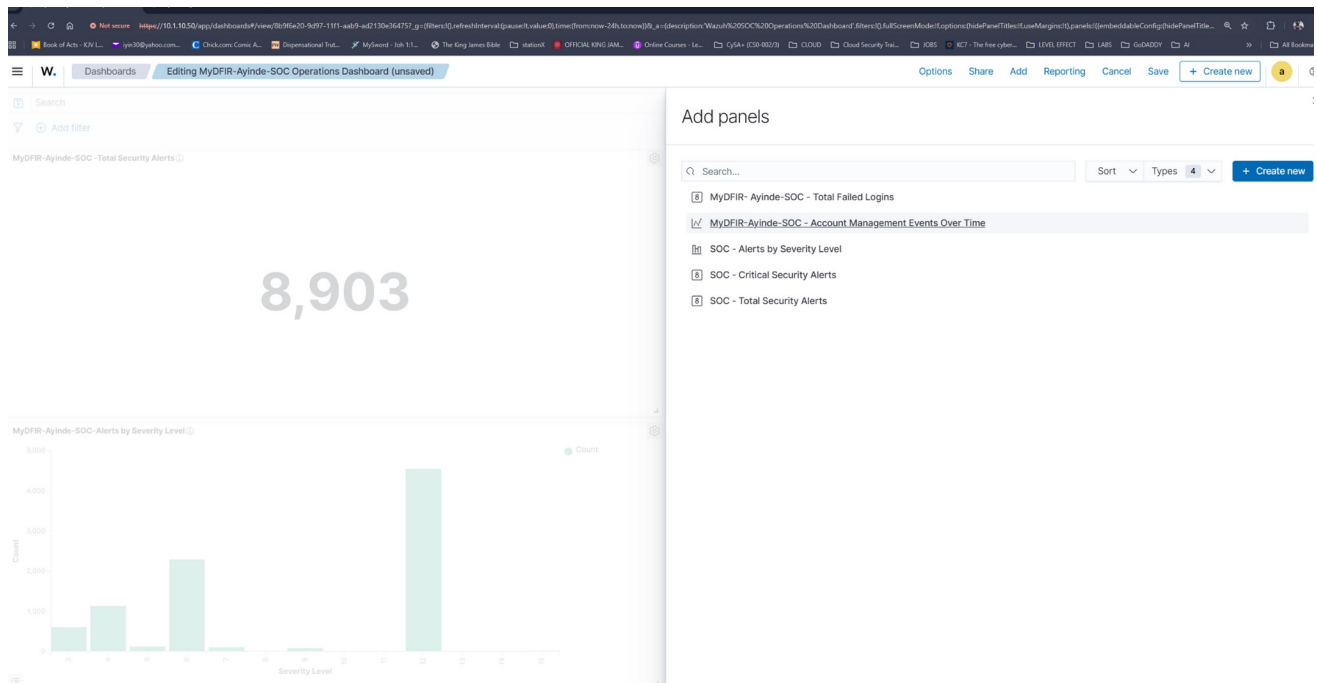


Figure 4B - Dashboard development view showing the custom visualization set being assembled.

Rule	Purpose	Level	Key mapping
100100	Windows Guest account enabled	12	T1098 - Account Manipulation / Persistence
100101	Member added to Remote Desktop Users group	12	T1098 - Account Manipulation / Persistence
100102	Multiple SSH login failures from the same source IP	10	T1110 - Brute Force / Credential Access

t id	1787542598.1548796
t input.type	log
t location	EventChannel
t manager.name	wazuh-srv-01
t rule.description	MyDFIR-Ayinde - Member added to Remote Desktop Users group
# rule.firedtimes	1
t rule.groups	windows, windows_security, group_changed, win_group_changed
t rule.id	100101
# rule.level	12
🔍 rule.mail	true
t rule.mitre.id	T1098
t rule.mitre.tactic	Persistence
t rule.mitre.technique	Account Manipulation
📅 timestamp	Aug 23, 2026 @ 23:36:38.960

Evidence EV-010 - Custom rule 100101 detected a member added to the Remote Desktop Users group; Wazuh recorded Level 12 and T1098 Account Manipulation.

Detection engineering lesson

Custom rules were built and tested against actual Wazuh field names and parent rules. AI can help draft detection logic, but the rule must be checked against Wazuh syntax, built-in rule relationships and live telemetry before it is trusted.

6. File Integrity Monitoring

Real-time FIM was configured on both Windows and Linux. Testing deliberately exercised file creation, modification and deletion so the analyst could observe how Wazuh records each state change and maps it to detection rules.

Rule	Observed event	Severity	Examples from lab
554	File added	Level 5	C:\CompanyData\payroll.txt; /opt/it-config/admin-settings.conf
550	Integrity checksum changed / file modified	Level 7	C:\CompanyData\payroll.txt; /opt/finance-data/budget.txt; /opt/app-config/appsettings.conf
553	File deleted	Level 7	C:\ITConfig\security.conf; /opt/it- config/security.conf

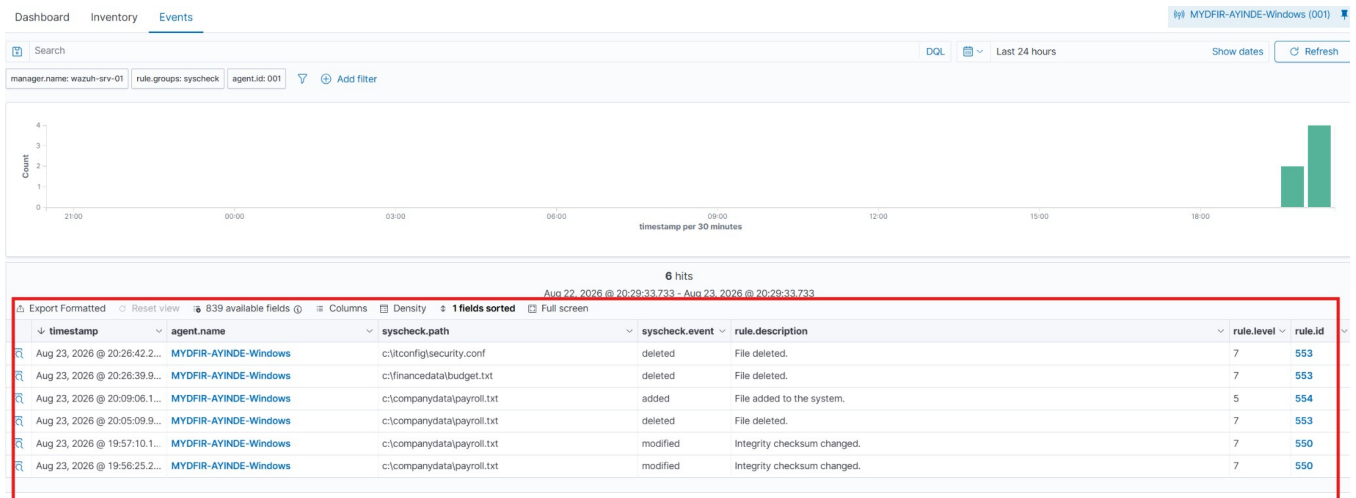


Figure 5 - Windows FIM event overview showing additions, modifications and deletions across monitored paths with rules 554, 550 and 553.

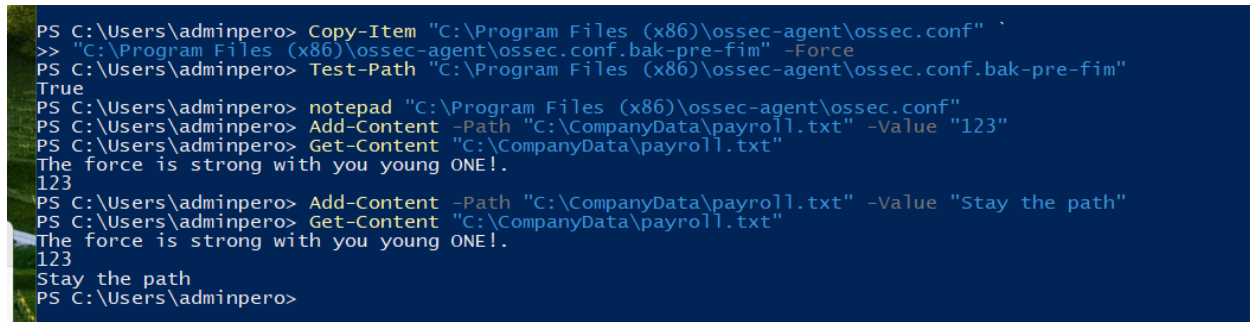
Not secure https://10.1.10.50/app/discover#/doc/wazuh-alerts-*/wazuh-alerts-4.x-2026

Book of Acts - KJV L... iyin30@yahoo.com... Chick.com: Comic A... Dispensational Trut... MySw

Discover wazuh-alerts-4.x-2026.08.23#HloNMaABjE3j0BFW5kHY

t agent.ip	10.1.10.51
t agent.name	MYDFIR-AVINDE-Windows
t decoder.name	syscheck_integrity_changed
t full_log	<pre> > File 'c:\companydata\payroll.txt' modified Mode: realtime Changed attributes: size,mtime,md5,sha1,sha256 Size changed from '42' to '47' Old modification time was: '1787528129', now it is '1787529380' Old md5sum was: 'd4c16d22fc7235ff97e7a515fedb7a72' New md5sum is: 'f6baf945f1da2245e04f08bb4f81d512' </pre>
t id	1787529385-23804701
t input.type	log
t location	syscheck
t manager.name	wazuh-srv-01
t rule.description	Integrity checksum changed.
# rule.firedtimes	1
t rule.gdpr	II_5.1.f
t rule.gpg13	4.11
t rule.groups	ossec, syscheck, syscheck_entry_modified, syscheck_file
t rule.id	550
# rule.level	/
rule.mail	false
t rule.mitre.id	T1565.001
t rule.mitre.tactic	Impact
t rule.mitre.technique	Stored Data Manipulation
t rule.nist_800_53	SI.7
t rule.pci_dss	11.5
t rule.tsc	PI1.4, PI1.5, CC6.1, CC6.8, CC7.2, CC7.3
t syscheck.attrs_after	ARCHIVE
t syscheck.changed_attributes	size, mtime, md5, sha1, sha256
t syscheck.event	modified
t syscheck.md5_after	f6baf945f1da2245e04f08bb4f81d512
t syscheck.md5_before	d4c16d22fc7235ff97e7a515fedb7a72
t syscheck.mode	realtime
syscheck.mtime_after	Aug 23, 2026 @ 19:56:20.000
syscheck.mtime_before	Aug 23, 2026 @ 19:35:29.000

Figure 5A - Wazuh FIM event detail used during validation of monitored file activity.



```

PS C:\Users\adminpero> Copy-Item "C:\Program Files (x86)\ossec-agent\ossec.conf" `
>> "C:\Program Files (x86)\ossec-agent\ossec.conf.bak-pre-fim" -Force
PS C:\Users\adminpero> Test-Path "C:\Program Files (x86)\ossec-agent\ossec.conf.bak-pre-fim"
True
PS C:\Users\adminpero> notepad "C:\Program Files (x86)\ossec-agent\ossec.conf"
PS C:\Users\adminpero> Add-Content -Path "C:\CompanyData\payroll.txt" -Value "123"
PS C:\Users\adminpero> Get-Content "C:\CompanyData\payroll.txt"
The force is strong with you young ONE!.
123
PS C:\Users\adminpero> Add-Content -Path "C:\CompanyData\payroll.txt" -Value "Stay the path"
PS C:\Users\adminpero> Get-Content "C:\CompanyData\payroll.txt"
The force is strong with you young ONE!.
123
Stay the path
PS C:\Users\adminpero>

```

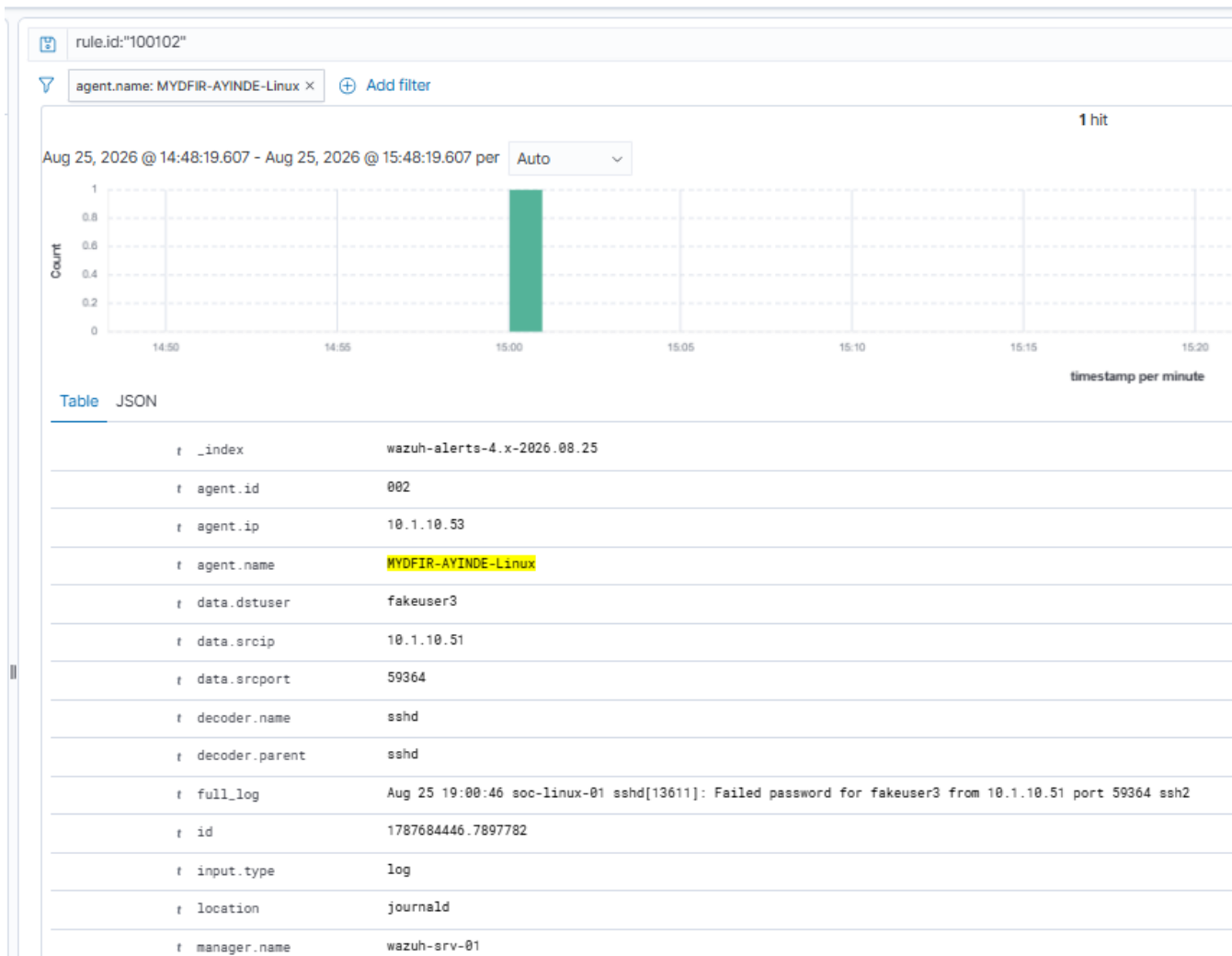
Figure 5B - Additional FIM validation evidence captured during the lab.

Attribution caution

FIM proves that a monitored file changed, was added or was deleted. File ownership metadata does not by itself prove which user or process performed the action. This distinction later became an explicit AI test-case requirement.

7. SSH Abuse Detection and Active Response

Repeated SSH failures were generated against soc-linux-01. Wazuh custom rule 100102 correlated three failed authentication attempts from the same source IP and classified the event as brute-force activity. Active Response then executed firewall-drop to block the source.



Evidence EV-011 - Failed SSH authentication event on soc-linux-01 showing source 10.1.10.51, destination user fakeuser3 and Linux agent ID 002.

	timestamp per second
data.command	add
data.dstuser	fakeuser3
data.origin.module	wazuh-execd
data.origin.name	node01
data.parameters.alert.agent.id	002
data.parameters.alert.agent.ip	10.1.10.53
data.parameters.alert.agent.name	MYOFIR-AYINDE-Linux
data.parameters.alert.data.dstuser	fakeuser3
data.parameters.alert.data.srcip	10.1.10.51
data.parameters.alert.data.srport	59364
data.parameters.alert.decoder.name	sshd
data.parameters.alert.decoder.parent	sshd
data.parameters.alert.full_log	Aug 25 19:00:46 soc-linux-01 sshd[13611]: Failed password for fakeuser3 from 10.1.10.51 port 59364 ssh2
data.parameters.alert.id	1787684446.7898855
data.parameters.alert.location	journalId
data.parameters.alert.manager.name	wazuh-srv-01
data.parameters.alert.predecoder.hostname	soc-linux-01
data.parameters.alert.predecoder.program_name	sshd
data.parameters.alert.predecoder.timestamp	Aug 25 19:00:46
data.parameters.alert.previous_output	Aug 25 19:00:41 soc-linux-01 sshd[13608]: Failed password for fakeuser3 from 10.1.10.51 port 59363 ssh2 Aug 25 19:00:40 soc-linux-01 sshd[13608]: Failed password for fakeuser3 from 10.1.10.51 port 59363 ssh2
data.parameters.alert.rule.description	MyOFIR-Ayinde - Multiple SSH login failures from same source IP
data.parameters.alert.rule.firedtimes	1
data.parameters.alert.rule.frequency	3
data.parameters.alert.rule.groups	syslog, sshd, authentication_failed
data.parameters.alert.rule.id	100102
data.parameters.alert.rule.level	10

Evidence EV-012 - Active Response context showing rule 100102, frequency 3 and the correlated SSH failures used to trigger firewall-drop.

Field	Value
Source IP	10.1.10.51
Destination host	soc-linux-01 / 10.1.10.53
Destination account	fakeuser3
Correlation rule	100102 - Multiple SSH login failures from same source IP
Rule level	10
MITRE	T1110 - Brute Force / Credential Access
Response	active-response/bin/firewall-drop
Response command	add
Block target	10.1.10.51

8. Investigation Structure + Collected Lab Evidence

The original project concludes by switching from event generation to analyst investigation. This report uses an investigation structure built around findings, an investigation summary, 5W1H analysis, recommendations and the collected lab evidence before documenting the AI enhancement.

Questions used to drive the investigation

What happened? When did it happen? Which hosts were involved? Which accounts were changed? Was a file deleted? Was SSH abuse observed? What should be recommended?

Investigation section	How this report applies it
Findings	Confirmed observations derived from Wazuh alerts, Security events, FIM telemetry and Active Response.
Investigation Summary	Chronological narrative connecting account changes, file activity, SSH abuse and automated response.
Who / What / When / Where / Why / How	Evidence-based 5W1H analysis, with uncertainty explicitly recorded where telemetry cannot answer a question.
Recommendations	Risk-reduction, validation and monitoring actions tied to the observed evidence.

Investigation standard

A confirmed event is not the same as a confirmed compromise. The report distinguishes observed facts, analyst interpretation and unknowns. This evidence discipline later became a core design requirement for the AI-assisted extension.

9. Findings

- Windows account-management telemetry showed the local Guest account being enabled and a member being added to the Remote Desktop Users group.
- Real-time FIM detected Windows and Linux file creation, modification and deletion events across monitored paths.
- Linux FIM recorded deletion of /opt/it-config/security.conf. The file owner was root, but the telemetry did not identify who performed the deletion.
- Repeated SSH authentication failures were observed against soc-linux-01 for fakeuser3 from 10.1.10.51.
- Custom rule 100102 correlated three SSH failures from the same source and mapped the behavior to T1110 Brute Force.
- Wazuh Active Response executed firewall-drop and blocked 10.1.10.51.
- No successful SSH authentication was observed in the investigated SSH sequence, and post-containment SSH activity was not observed in the supplied evidence.
- The lab activity was generated for controlled testing; the investigation treats the telemetry as an analyst case and avoids assuming malicious intent where evidence is incomplete.

Finding discipline

Observed facts, interpretations and unknowns are kept separate. This improves defensibility of both the human investigation and the later AI evaluation.

10. Investigation Summary and Master Evidence Timeline

The investigation identified a sequence of security-relevant activity across the monitored Windows and Linux endpoints. Account-management changes and FIM events occurred on Aug 23, followed by repeated SSH authentication failures and automated containment on Aug 25. Exact timestamps captured from Wazuh were used to build the evidence timeline.

ID	Local time (UTC-4)	Host	Artifact / event	Rule / mapping
EV-001	Aug 23 19:56:25.211	LABSWIN-11-01	C:\CompanyData\payroll.txt modified	550 / T1565.001
EV-002	Aug 23 20:09:06.110	LABSWIN-11-01	C:\CompanyData\payroll.txt added	554
EV-003	Aug 23 20:26:42.294	LABSWIN-11-01	C:\ITConfig\security.conf deleted	553 / T1070.004, T1485
EV-004	Aug 23 21:53:57.884	soc-linux-01	/opt/company-data/test.txt modified	550
EV-005	Aug 23 21:56:52.626	soc-linux-01	/opt/it-config/admin-settings.conf added	554
EV-006	Aug 23 22:01:00.896	soc-linux-01	/opt/finance-data/budget.txt modified	550
EV-007	Aug 23 22:01:02.923	soc-linux-01	/opt/app-config/appsettings.conf modified	550
EV-008	Aug 23 22:01:04.950	soc-linux-01	/opt/it-config/security.conf deleted	553 / T1070.004, T1485
EV-009	Aug 23 23:05:42.094	LABSWIN-11-01	Guest account enabled	100100 / Event 4722 / T1098
EV-010	Aug 23 23:36:38.960	LABSWIN-11-01	Member added to Remote Desktop Users	100101 / Event 4732 / T1098
EV-011	Aug 25 15:00:40-46	soc-linux-01	Three failed SSH logins from 10.1.10.51	5760 -> 100102 / T1110
EV-012	Aug 25 15:00:48.609	soc-linux-01	Active Response firewall-drop executed	wazuh-execd / block 10.1.10.51

Timestamp note

The Wazuh dashboard displayed local lab time (UTC-4). Underlying alert payloads for the SSH sequence also contained UTC timestamps, e.g. 19:00:46Z corresponding to 15:00:46 local.

11. Who, What, When, Where, Why and How

Question	Assessment
Who	Accounts/identities visible in telemetry included adminpero, Guest and fakeuser3. The Remote Desktop Users event contained a member SID that was not resolved to a friendly username. These identities are evidence subjects, not automatically confirmed malicious actors.
What	Account enablement, remote-access group membership change, monitored file additions/modifications/deletions, repeated SSH authentication failures and automated firewall-drop containment were observed.
When	The principal evidence spans Aug 23-25, 2026. Exact local timestamps are recorded in the Master Evidence Timeline, with UTC correlation noted for SSH alerts.
Where	Windows activity occurred on LABSWIN-11-01 (10.1.10.51); Linux FIM and SSH activity occurred on soc-linux-01 (10.1.10.53); correlation and response were handled by wazuh-srv-01 (10.1.10.50).
Why	The telemetry does not independently establish motive or authorization for every event. In the controlled lab, actions were intentionally generated for detection/investigation testing. In a real incident, authorization would need to be validated.
How	Wazuh collected Windows Security, Sysmon, Linux journald and FIM telemetry. Built-in and custom rules generated alerts; rule 100102 correlated repeated SSH failures; Active Response invoked firewall-drop to block the source IP.

12. Recommendations

Area	Recommended action	Rationale
Identity	Keep Guest disabled unless there is a documented business requirement; alert on unexpected enablement.	Guest enablement can change local access exposure and should be validated.
Remote access	Review membership of Remote Desktop Users and privileged groups; require approved change records.	Membership changes can expand remote-access capability.
SSH	Use key-based authentication where practical, restrict source networks and review repeated failures.	Reduces password-guessing exposure and improves attribution.
FIM	Continue real-time monitoring of security and application configuration paths; investigate unexpected deletions.	Provides early visibility into configuration tampering and deletion.
Detection	Retain and tune custom Wazuh rules; validate fields and parent-rule relationships after Wazuh upgrades.	Custom detections must remain aligned with actual telemetry.
Response	Use Active Response for well-tested conditions and keep rollback/recovery verification steps.	Automated containment reduces response time but must be controlled.
Investigation	Preserve raw archives and exact timestamps; separate observed facts from assumptions.	Improves defensibility and repeatability of investigations.
Operations	Use human review before consequential remediation when context is incomplete.	Prevents over-response based on ambiguous telemetry.

13. Project Enhancement - AI-Assisted SOC Triage

After the original Wazuh investigation was completed, the lab was extended to evaluate whether a local AI model could assist a Tier 1 analyst with evidence interpretation, classification, missing-context identification, recommended actions and executive summaries. The enhancement remained local using Ollama so the design could be evaluated without relying on a paid cloud API.

What changed

The original project ended at analyst investigation and reporting. The enhancement added structured JSON test cases, human ground truth, a SOC playbook, Python policy engine, local Ollama/Qwen models, output validator, evaluator and consolidated benchmark. The original Wazuh evidence remained the source of truth.

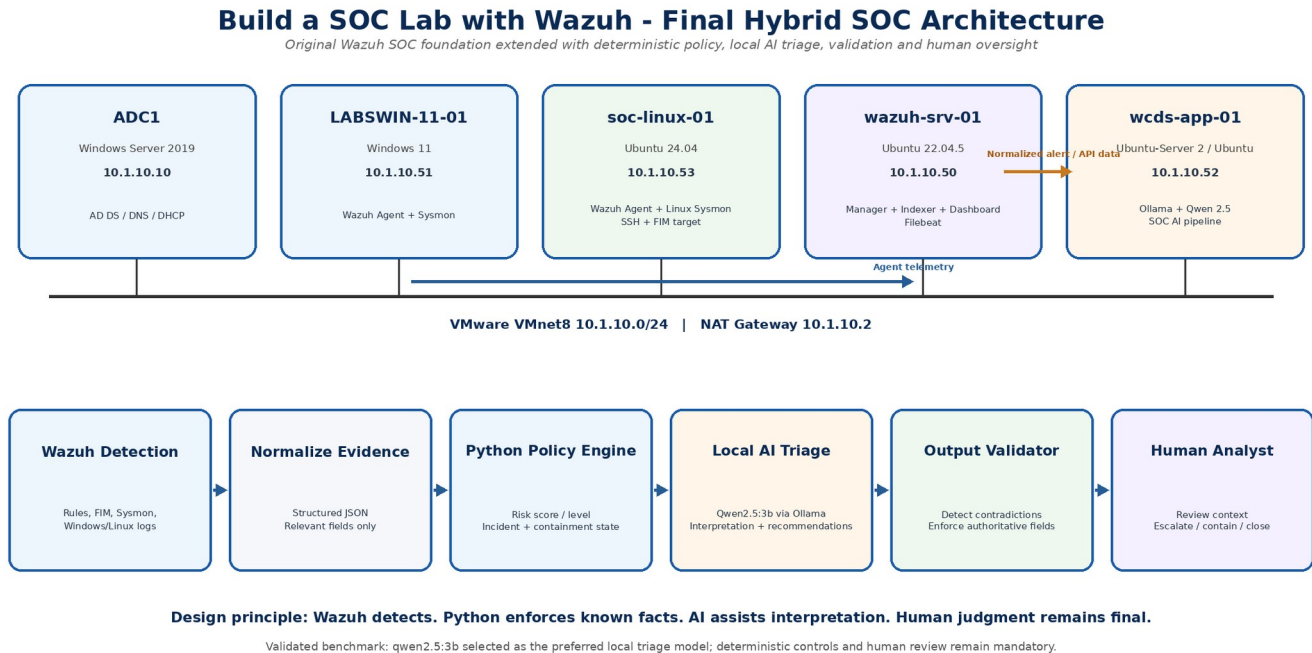


Figure 6 - Final hybrid architecture with corrected system roles, IP addresses and a readable Wazuh -> policy -> local AI -> validator -> human analyst workflow.

14. AI Implementation Progression

1. Created a reusable ~/soc-ai project structure with test-cases, ground-truth, playbook, results and runner directories.
2. Converted real Wazuh evidence into normalized JSON test cases and created separate human analyst ground-truth files.
3. Built build_prompt.py to combine the SOC playbook, test-case evidence and verified policy without exposing the human answer key to the model.
4. Installed Ollama locally on wcds-app-01 and benchmarked qwen2.5:1.5b and qwen2.5:3b in CPU-only mode.
5. Observed that raw LLM output could classify events reasonably but was unreliable for deterministic risk arithmetic and some incident-state fields.
6. Built policy_engine.py to calculate authoritative risk, incident and containment state using deterministic logic.
7. Updated the prompt so the LLM could interpret authoritative values but was not responsible for calculating them.
8. Built output_validator.py to compare AI output with policy state, correct contradictions and preserve an audit trail.
9. Built evaluator.py v1.1 to separate hard Policy Accuracy from softer Analyst Agreement.
10. Built benchmark_summary.py and generated a consolidated four-case, two-model benchmark.

```
osboxes@wcds-app-01:~$ ollama list
NAME          ID          SIZE      MODIFIED
qwen2.5:1.5b  65ec06548149  986 MB    2 minutes ago
osboxes@wcds-app-01:~$ ollama run qwen2.5:1.5b "Respond with only this JSON: {\"status\":\"SOC AI ready\"}"
{
  "status": "SOC AI ready"
}
osboxes@wcds-app-01:~$
```

Figure 7 - Local Ollama/Qwen validation: qwen2.5:1.5b returned structured JSON successfully before the full SOC pipeline was connected.

15. AI Issues, Troubleshooting and Design Resolutions

Issue	Observed behavior	Engineering resolution
AI-01 - Risk arithmetic	1.5B returned 0/Low; 3B returned 30/Medium for the SSH case instead of 60/High.	Moved risk calculation into policy_engine.py: Level 10 = 40, repeated authentication = +20, total = 60/High.
AI-02 - Compromise inference	Models sometimes returned Observed or Not Observed when the evidence only supported Unknown.	Added deterministic incident_state fields and enforced them through the validator.
AI-03 - Containment contradiction	1.5B returned already_triggered=false even though firewall-drop had executed.	Output validator compared AI containment fields to the policy result and corrected conflicts.
AI-04 - CPU inference timeout	qwen2.5:3b exceeded the original 600-second runner timeout on the long SOC prompt.	Validated the model with a small prompt, then increased the client timeout to 1800 seconds while keeping benchmark inputs unchanged.
AI-05 - Classification variability	Both models were weaker on precise FIM classification; 1.5B also used generic Suspicious labels for account/access cases.	Separated Policy Accuracy from Analyst Agreement and retained mandatory human review instead of forcing all differences into hard corrections.

```
1.5B baseline preserved
osboxes@wcds-app-01:~$ python3 ~/soc-ai/runner/run_ollama.py qwen2.5:3b
Model: qwen2.5:3b
Sending Test Case 001 to local SOC AI...
Runner error: timed out
osboxes@wcds-app-01:~$
```

Issue AI-04 - qwen2.5:3b exceeded the initial 600-second timeout on CPU-only inference.

```

osboxes@wcds-app-01:~$ nano ~/soc-ai/runner/policy_engine.py
osboxes@wcds-app-01:~$ python3 -m py_compile ~/soc-ai/runner/policy_engine.py
osboxes@wcds-app-01:~$ echo $?
0
osboxes@wcds-app-01:~$ python3 ~/soc-ai/runner/policy_engine.py
{
  "alert_id": "100102-20260825-150046",
  "policy_version": "1.0",
  "validation": {
    "missing_required_fields": [],
    "input_valid": true
  },
  "risk": {
    "wazuh_rule_level": 10,
    "base_score": 40,
    "modifiers": [
      {
        "reason": "Repeated authentication failures",
        "points": 20
      }
    ],
    "risk_score": 60,
    "risk_level": "High"
  },
  "containment": {
    "active_response_triggered": true,
    "action": "firewall-drop",
    "command": "add",
    "source_ip_blocked": "10.1.10.51"
  }
}

Policy result saved: /home/osboxes/soc-ai/results/SOC-AI-Test-Case-001-policy.json
osboxes@wcds-app-01:~$

```

Resolution AI-01 - Deterministic policy engine produced the authoritative SSH score of 60/High and recorded the active firewall-drop state.

16. AI Test Cases and Human Ground Truth

Test case	SOC domain	Evidence used	Expected analyst interpretation
TC-001	SSH brute force / authentication	Rule 100102, 3 failed logins, fakeuser3, source 10.1.10.51, firewall-drop	Brute Force Attempt; 60/High; no successful authentication observed; containment triggered.
TC-002	Windows account management	Event 4722, Guest enabled, rule 100100 Level 12	Account Manipulation; 70/High; authorization requires validation; compromise state Unknown.
TC-003	Access control / group membership	Event 4732, Remote Desktop Users, unresolved member SID, rule 100101	Access Modification; 75/High; do not invent SID identity; compromise state Unknown.
TC-004	Linux FIM	/opt/it-config/security.conf deleted, rule 553 Level 7, root ownership metadata	File Deletion; 40/Medium; actor identity Unknown; do not infer root performed deletion.

Ground-truth discipline

The model never received the human ground-truth file during inference. Ground truth was used only after the run to evaluate the validated AI result.

17. AI Benchmark Results

Eight benchmark runs were completed: four SOC scenarios against two local models. The final comparison separates post-validation policy correctness from analyst-style agreement. This avoids presenting the validator-enforced 100% figure as raw model accuracy.

Model	Validated Policy Accuracy	Average Analyst Agreement	Validator corrections	Exact classifications
qwen2.5:1.5b	100.0%	45.0%	4	1 / 4
qwen2.5:3b	100.0%	61.25%	3	2 / 4

Case	1.5B Analyst Agreement	3B Analyst Agreement	Interpretation
TC-001 - SSH Brute Force	100%	65%	1.5B matched the human classification more closely; 3B was more conservative.
TC-002 - Guest Enablement	20%	80%	3B performed substantially better on account-management classification.
TC-003 - RDP Group Change	40%	80%	3B performed better on access-control classification.
TC-004 - FIM File Deletion	20%	20%	Both models showed weak precision on the FIM analyst classification.

```

osboxes@wcds-app-01:~$ ~/soc-ai/results/SOC-AI-Consolidated-Benchmark.json
bash: /home/osboxes/soc-ai/results/SOC-AI-Consolidated-Benchmark.json: No such file or directory
osboxes@wcds-app-01:~$ nano ~/soc-ai/runner/benchmark_summary.py
osboxes@wcds-app-01:~$ python3 -m py_compile ~/soc-ai/runner/benchmark_summary.py
echo $?
0
osboxes@wcds-app-01:~$ python3 ~/soc-ai/runner/benchmark_summary.py
Consolidated SOC AI benchmark generated.
Benchmark runs processed: 8

Model: qwen2.5:1.5b
Average Policy Accuracy: 100.0%
Average Analyst Agreement: 45.0%
Validator Corrections: 4
Exact Classification Matches: 1

Model: qwen2.5:3b
Average Policy Accuracy: 100.0%
Average Analyst Agreement: 61.25%
Validator Corrections: 3
Exact Classification Matches: 2

Consolidated benchmark saved: /home/osboxes/soc-ai/results/SOC-AI-Consolidated-Benchmark.json
osboxes@wcds-app-01:~$ python3 -m json.tool \
~/soc-ai/results/SOC-AI-Consolidated-Benchmark.json \
> /dev/null && \
echo "Consolidated SOC AI benchmark valid"
Consolidated SOC AI benchmark valid
osboxes@wcds-app-01:~$

```

Figure 8 - Automated consolidated benchmark generation: 8 runs processed; both models reached 100% validated policy accuracy, while 3B achieved higher average analyst agreement and fewer corrections.

Model selection

qwen2.5:3b was selected as the preferred local Tier 1 triage model because it achieved higher average analyst agreement (61.25%) and required fewer validator corrections (3 vs 4). It remains advisory and requires deterministic controls plus human review.

18. Final Operational Model

Layer	Responsibility	Authority
Endpoints	Generate Windows/Linux security telemetry and FIM events.	Observed telemetry
Wazuh	Collect, correlate, detect, alert and execute configured Active Response.	Security monitoring platform / SIEM function
Normalizer	Convert selected alert fields into structured JSON evidence.	Data preparation
Python Policy Engine	Calculate risk; define known incident and containment state.	Authoritative deterministic control
Ollama + Qwen 2.5 3B	Interpret evidence, classify activity, identify missing context and recommend analyst actions.	Advisory AI
Output Validator	Detect AI contradictions and enforce policy fields while recording corrections.	Quality gate
Human Analyst	Review context, approve escalation/containment and determine final disposition.	Final decision authority

Operational principle

Wazuh detects. Python enforces known facts. AI explains and recommends. Human judgment remains final.

19. Lessons Learned and Skills Demonstrated

Area	Demonstrated capability
SIEM / Wazuh	Deployment, agent integration, archive hunting, dashboard development, custom rules and Active Response.
Windows telemetry	Windows Security events, Sysmon, account-management detections and local group-change analysis.
Linux telemetry	journald/sshd analysis, Linux Sysmon, FIM and SSH authentication investigation.
Detection engineering	Custom rule development, parent-rule validation, field mapping, MITRE ATT&CK context and test verification.
Incident response	Evidence collection, timeline building, containment validation, recovery checks and recommendations.
Python / JSON	Prompt builder, deterministic policy engine, output validator, evaluator and consolidated benchmark automation.
Local AI / LLM evaluation	Ollama deployment, Qwen model testing, CPU performance troubleshooting, human ground truth and model benchmarking.
AI governance	Evidence grounding, uncertainty handling, human-in-the-loop design and deterministic control of consequential security state.

Core lesson

The strongest result was not that an AI model could produce a SOC narrative. It was the discovery, through testing, that deterministic controls and human review are necessary to make AI-assisted security triage defensible.

20. Evidence and Artifact Index

Artifact / evidence	Purpose
SOC-AI-Test-Case-001..004.json	Normalized Wazuh evidence used as AI inputs.
SOC-AI-Test-Case-001..004-ground-truth.json	Independent human analyst answer keys; not exposed to the model during inference.
policy_engine.py	Deterministic risk and incident-state policy engine.
build_prompt.py	Builds the AI prompt from playbook, evidence and authoritative policy.
run_ollama.py	Reusable local model runner supporting multiple models and test cases.
output_validator.py	Compares AI output to policy and records/corrects authoritative conflicts.
evaluator.py v1.1	Separates hard Policy Accuracy from soft Analyst Agreement.
benchmark_summary.py	Consolidates eight benchmark results automatically.
SOC-AI-Consolidated-Benchmark.json	Machine-readable cross-model benchmark summary.
SOC-AI-Benchmark-Summary.md	Human-readable benchmark summary suitable for GitHub.
Wazuh screenshots	Evidence of deployment, dashboards, FIM, custom detection rules, SSH failures, Active Response and investigation.
Network / architecture diagrams	Document original Wazuh topology and final hybrid SOC architecture.

Suggested GitHub layout: README.md for the narrative; docs/ for the PDF and diagrams; runner/ for Python scripts; test-cases/ and ground-truth/ for evaluation evidence; results/ for validated benchmark artifacts; screenshots/ for labelled evidence figures.

Screenshot coverage

The PDF includes selected evidence for each major phase - deployment/integration, dashboards, custom detection, FIM, Active Response, investigation and AI validation - while avoiding a repetitive command-by-command screenshot dump.

21. References & Credits

Technical references used to support the lab implementation, detection logic, investigation terminology and AI-assisted extension are listed below. Lab screenshots and benchmark results are original evidence captured during the build.

[1] MyDFIR (@Steve). Original project inspiration - Build a SOC Lab with Wazuh series. [Reference link](#)

[2] Wazuh Documentation. File Integrity Monitoring. [Reference link](#)

[3] Wazuh Documentation. Custom Rules. [Reference link](#)

[4] Wazuh Documentation. Active Response. [Reference link](#)

[5] MITRE ATT&CK. T1110 - Brute Force. [Reference link](#)

[6] MITRE ATT&CK. T1098 - Account Manipulation. [Reference link](#)

[7] MITRE ATT&CK. T1070.004 - File Deletion. [Reference link](#)

[8] MITRE ATT&CK. T1485 - Data Destruction. [Reference link](#)

[9] Ollama Documentation. Generate API used by the local SOC AI runner. [Reference link](#)

[10] Ollama Model Library. Qwen2.5 / qwen2.5:3b. [Reference link](#)

Source note: This report also uses the supplied project overview, investigation structure, SOC playbook, FIM/custom-rule reference material and collected Wazuh/terminal evidence from the lab. All IP addresses shown are private lab addresses.